# CS100 Fall 2023

Quiz 2

Dec 28, 2023

Answer the questions according to the C++17 standard.

1. (15 points) Your name: _____. Your student ID: _____.
   Your email: _____@shanghaitech.edu.cn

2. The following is a word-counting program. It reads its input and reports how often each word appears.

```
std::map<std::string, std::size_t> word_count;
std::string word;
while (std::cin >> word)
  ++word_count[word];
for (const auto &[word, occr] : word_count)
  std::cout << word << " occurs " << occr
            << (occr == 1 ? "time" : "times") << std::endl;
```

   Based on your understanding of the standard library, answer the following questions.

   (a) (5 points) `word_count` maps each word to a counter, which is an integer indicating its number of its occurrences. `++word_count[word]` increments the counter for the word `word`, but we did not set the initial value for that counter explicitly. What is the initial value of it?

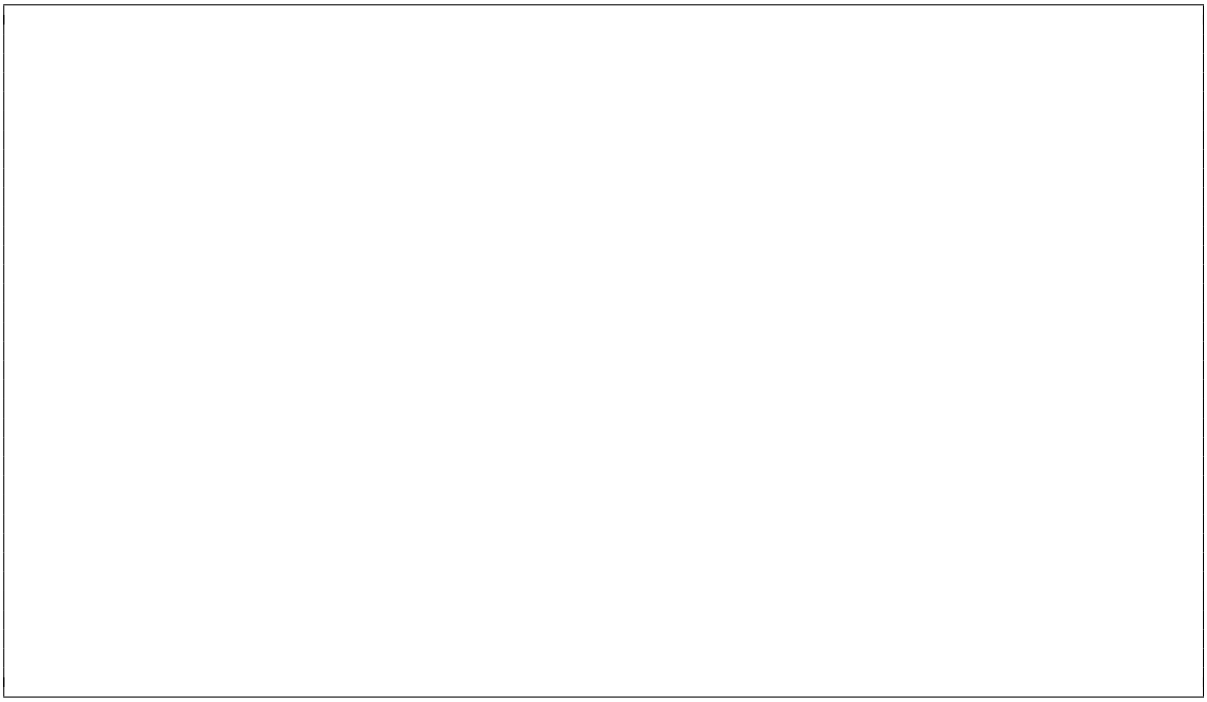
   (b) (10 points) We have learned the `const`-vs-non-`const` overloads for the subscript operator of some simple containers like `std::vector` and `Dynarray`. Does `std::map<S, T>::operator[]` have a `const` version? Why?

3. (30 points) Given the following classes `Item` and `DiscountedItem`

```
class Item {
protected:
  std::string mName;
  double mPrice;
public:
  Item(std::string name, double price) : mName{std::move(name)}, mPrice{price} {}
  virtual double netPrice(unsigned cnt) const { return mPrice * cnt; }
  virtual ~Item() = default;
};
class DiscountedItem : public Item {
protected:
  unsigned mMinQuantity;
  double mDiscount;
public:
  DiscountedItem(std::string name, double price, unsigned minQuantity, double discount)
      : Item{std::move(name), price}, mMinQuantity{minQuantity}, mDiscount{discount} {}
  double netPrice(unsigned cnt) const override {
    return cnt < mMinQuantity
          ? mPrice * cnt
          : mPrice * mMinQuantity + mPrice * mDiscount * (cnt - mMinQuantity);
  }
};
```

Define a class `LimitedDiscountItem` (you can write it as `LDI`) that implements a *limited discount strategy*, which applies a discount to items purchased up to a given limit. If the number of items purchased exceeds that limit, the normal price applies to those purchased beyond the limit.

Will you make `LimitedDiscountItem` inherit from `Item`? `DiscountedItem`? Or, design a completely different inheritance hierarchy? Write your code and explain your design on this. Your code should include at least the data members of `LimitedDiscountItem` and its `netPrice` function.



4. A class can also inherit constructors from its base class. For example,

```
class BinaryOp : public Expression {
protected:
  Node left, right;
public:
  BinaryOp(Node l, Node r) : Expression{}, left{std::move(l)}, right{std::move(r)} {}
  // ...
};
class PlusOp : public BinaryOp {
  using BinaryOp::BinaryOp;
  // ...
};
```

By declaring `using BinaryOp::BinaryOp;`, the class `PlusOp` has a constructor inherited from `BinaryOp` which has the similar effect as

```
  PlusOp(Node l, Node r) : BinaryOp(std::move(l), std::move(r)) {}
```

that is, it accepts the same parameters and initializes the base class subobject in the same way as the base class's constructor. Based on your understanding about classes and inheritance, answer the following questions.

(a) (10 points) The `using` declaration is written in a `class`, without an explicitly written access specifier. What is the access level of the inherited constructor (public, private, or protected)? Explain your idea.

(b) (10 points) If `PlusOp` also has its own data members, how are they initialized in the inherited constructor?

5. Both `std::string` and `std::vector` defined an overloaded `==` that can be used to compare objects of those types. Assuming `svec1` and `svec2` are `vector`s that hold `string`s, identify which version of `==` is applied in each of the following expressions.

   A. The `==` for `std::string`s.

   B. The `==` for `std::vector`s.

   C. Neither A nor B, but the expression is well-formed.

   D. Neither A nor B, and it is an error.

   (a) (5 points) `"cobble" == "stone"`

   (a) _____

   (b) (5 points) `svec1[0] == svec2[0]`

   (b) _____

   (c) (5 points) `svec1 == svec2`

   (c) _____

   (d) (5 points) `svec1[0] == "stone"`

   (d) _____